

HelixQA

HelixQA

एंटी-ब्लफ़ क्यूए ऑर्किस्ट्रेशन — स्वायत्त, क्रॉस-प्लेटफ़ॉर्म सत्र जहाँ हर 'पास' में पकड़ा गया ऐसा प्रमाण होता है जिसे कोई वास्तविक उपयोगकर्ता फ़ीचर का उपयोग कर सकता है।

सारांश

HelixQA एक एंटी-ब्लफ़ क्यूए ऑर्किस्ट्रेशन फ्रेमवर्क है जो क्रॉस-प्लेटफ़ॉर्म परीक्षण (एंड्रॉइड, एंड्रॉइड टीवी, वेब, डेस्कटॉप) के लिए है। यह YAML टेस्ट बैंकों, रीयल-टाइम क्रैश डिटेक्शन, चरण-दर-चरण प्रमाण संग्रहण और LLM-प्लस-कंप्यूटर-विज्ञान स्वायत्त क्यूए सत्रों को जोड़ता है ताकि यह साबित किया जा सके कि फ़ीचर वास्तव में एंड-टू-एंड काम करते हैं। यह Constitution का अनिवार्य क्यूए टेस्ट-टाइप (§11.4.169) है।

संक्षिप्त विवरण

एक एंटी-ब्लफ़ क्यूए ऑर्किस्ट्रेटर (Go) जो लिखित टेस्ट बैंकों और पूर्णतः स्वायत्त, LLM-और-विज्ञान-चालित क्यूए सत्रों को प्लेटफ़ॉर्मों पर चलाता है — क्रैश का पता लगाता है, हर चरण को पकड़े गए प्रमाणों (स्क्रीनशॉट, लॉगकैट, वीडियो, स्टैक ट्रेस) के विरुद्ध सत्यापित करता है, और AI फिक्स पाइपलाइनों के लिए प्रमाण-समृद्ध टिकट स्वतः उत्पन्न करता है।

विस्तृत विवरण

HelixQA एक Go फ्रेमवर्क है जिसका एकमात्र, अडिग डिज़ाइन केंद्र Constitution का §11.4 संचालन नियम है: शिपिंग का मानक "टेस्ट पास" नहीं, बल्कि "उपयोगकर्ता फ़ीचर का उपयोग कर सकते हैं" है, इसलिए इसके द्वारा जारी हर 'पास' में निष्पादन के दौरान पकड़ा गया सकारात्मक रनटाइम प्रमाण होना चाहिए — कोई प्रमाण नहीं, कोई ग्रीन नहीं, कोई अपवाद नहीं। यह दो पूरक मोड चलाता है जो मिलकर स्क्रिप्टेड और अज्ञात दोनों को कवर करते हैं। पहला, **लिखित टेस्ट बैंक** — YAML सूट जिनमें TC-XXX केस, प्लेटफ़ॉर्म टारगेटिंग, प्राथमिकता, क्रमबद्ध चरण (नाम/क्रिया/अपेक्षित), टैग और दस्तावेज़ संदर्भ होते हैं — जिन्हें प्रति-चरण सत्यापन, रीयल-टाइम क्रैश/एएनआर डिटेक्शन (एंड्रॉइड के लिए एडीबी, वेब/डेस्कटॉप के लिए प्रोसेस मॉनिटरिंग), केंद्रीकृत प्रमाण संग्रहण और डाउनस्ट्रीम AI फिक्स पाइपलाइनों के लिए पहले से तैयार मार्कडाउन टिकटों के साथ निष्पादित किया जाता है। दूसरा, **पूर्णतः स्वायत्त क्यूए सत्र** जो ऐप को LLM-संचालित एजेंटों और कंप्यूटर विज्ञान के हवाले कर देता है और उन्हें चार अनुशासित चरणों में बिना निगरानी के चलाने देता है: सेटअप (एलएलएम चुनना, प्रोजेक्ट दस्तावेज़ों से फ़ीचर मैप

बनाना, CLI एजेंटों को स्पॉन करना, विज्ञान इंजन आरंभ करना), दस्तावेज़-चालित सत्यापन जो हर दर्ज फ़ीचर की जाँच करता है, जिज्ञासा-चालित अन्वेषण जो जानबूझकर किनारे के मामलों और अदस्तावेज़ी व्यवहार को टटोलता है, फिर मार्कडाउन/HTML/JSON में रिपोर्ट-और-सफाई, जहाँ हर खोज वीडियो-टाइमस्टैम्प प्रमाण से जुड़ी होती है।

महत्वपूर्ण बात यह है कि यह अपना खुद का होमवर्क ग्रेड नहीं करता: यह चार बाहरी Go सबमॉड्यूल (LLMsVerifier, LLMOrchestrator, VisionEngine, DocProcessor) को एकीकृत करता है और साझा चैलेंज और कंटेनर इंफ्रास्ट्रक्चर का पुनः उपयोग करता है, ताकि जो घटक ऐप को नेविगेट करता है वह घटक न हो जो यह निर्णय ले कि यह काम किया या नहीं। इसका अपना सूट ठीक उसी मानक पर खरा उतरता है जिसे यह दूसरों पर लागू करता है — make anti-bluff (स्टैटिक स्कैन + व्यवहार-एंकर मेनिफेस्ट + म्यूटेशन रैचैट) और एक 8-चरण ऑर्केस्ट्रेटर चैलेंज के माध्यम से, जिसमें अंतर्निहित §1.1 म्यूटेशन होता है। एक 15-पंक्ति टेस्ट-टाइप कवरेज मैट्रिक्स हर विज्ञापित क्षमता को एक ठोस निष्पादन योग्य संपत्ति और एक विशिष्ट पकड़े गए प्रमाण के रूप से जोड़ता है — ताकि फ्रेमवर्क के अपने दावे भी उतने ही प्रमाण-बद्ध हों जितने कि वे निर्णय जो यह उन उत्पादों के लिए देता है जिनका यह परीक्षण करता है।

सामग्री

हमने इसे क्यों बनाया

पारंपरिक क्वालिटी एश्योरेंस (QA) तब “पुष्टि सफल” का हरा संकेत दे देता है, जबकि Constitution जिसे धोखेबाज़ी कहता है—वह गड़बड़ी इसी तरह फिसल जाती है। यानी एक फीचर को काम करता हुआ रिपोर्ट किया जाता है, जबकि असल उपयोगकर्ता के लिए वह टूटा होता है। HelixQA को खास तौर पर QA के लिए इसी समस्या को असंभव बनाने के लिए बनाया गया है: यह बिना वास्तविक सबूत (स्क्रीनशॉट, लॉगकैट, वीडियो, स्टैक ट्रेस, रिपोर्ट) के PASS स्कोर नहीं देता, जो वास्तविक परीक्षण के दौरान कैप्चर किया गया हो। साथ ही, बिना सबूत के हरे रंग की सारांश लाइन को भी एक गंभीर दोष मानता है, जो किसी गायब फीचर के बराबर होता है। यह श्रम की समस्या का भी समाधान करता है—कई प्लेटफार्मों पर व्यापक मैनुअल QA को बढ़ाना संभव नहीं होता—क्योंकि यह सत्रों को पूरी तरह स्वायत्त बना देता है।

यह क्यों गेम-चेंजर है

यह दो ऐसी चीज़ों को जोड़ता है जो लगभग कभी एक ही ढूल में नहीं होतीं: कठोर, सबूत-आधारित QA गेटिंग और स्वायत्त, स्वचालित अन्वेषण। एक LLM-प्लस-विज्ञान एजेंट वास्तविक ऐप खोलता है, हर दस्तावेज़ीकृत फीचर की पुष्टि करता है, उन अनदस्तावेज़ी बग्स की तलाश करता है जिनके लिए किसी ने टेस्ट नहीं लिखा होता, और इसके साथ-साथ अदालती गुणवत्ता का सबूत भी तैयार करता है—ताकि “हमने इसका परीक्षण किया” की जगह “यह रहा वीडियो, यह रहा लॉगकैट, यह रहा टिकट” आ जाए। और चूँकि यह Constitution-नामित QA सबमॉड्यूल है, इसलिए इसे अपनाने से एक टीम के QA की ईमानदारी नहीं बढ़ती—बल्कि एक ही कदम में पूरे उत्पाद परिवार के लिए न्यूनतम मानक ऊँचा हो जाता है।

क्या है नवाचारी

- **धोखेबाजी-रोधी सबूत अनुबंध** — हर जाँच का PASS कैप्चर किए गए रनटाइम सबूत से बंधा होता है; हरी CI लाइन को जरूरी तो माना जाता है, लेकिन कभी पर्याप्त नहीं, और बिना सबूत के हरे सारांश को गंभीर दोष के रूप में स्कोर किया जाता है।
- **स्वायत्त दस्तावेज़-चालित + जिज्ञासा-चालित अन्वेषण** — यह हर दस्तावेज़ीकृत फीचर की पुष्टि करता है और फिर स्क्रिप्ट से हटकर उन किनारे के मामलों की जाँच करता है जिनसे असली उपयोगकर्ता टकराते हैं (खाली इनपुट, तेज़ इंटरैक्शन, अनदस्तावेज़ रास्ते) जिन्हें कोई हस्तलिखित टेस्ट सुइट नहीं पकड़ पाता।
- **विज्ञान ओरेकल** — GoCV मैकेनिकल विज्ञान और LLM विज्ञान API सचमुच स्क्रीन पर चल रहे UI को देखता है, उन विज़ुअली टूटे हुए स्टेट्स को पकड़ता है जिन्हें टोकन-और प्रॉपर्टी-स्तर की पुष्टियाँ सीधे नज़रअंदाज़ कर देती हैं।
- **संरचना-आधारित टेस्ट बैंक** — बैंक स्ट्रिंग्स संरचना का वर्णन करती हैं और रनटाइम पर LLM-जनित प्रश्न प्रॉम्प्ट्स को संचालित करती हैं (CONST-046), ताकि एक ही बैंक सभी लोकेशनों पर काम करे, बजाय इसके कि UI टेक्स्ट के अनुवाद होते ही वह बिखर जाए।
- **AI फिक्स पाइपलाइनों के लिए तैयार टिकट** — स्वतः जनित मार्कडाउन इश्यूज़ पूरे सबूत बंडल के साथ आते हैं, जिन्हें सीधे डाउनस्ट्रीम रिपेयर एजेंट को सौंपा जा सकता है, न कि किसी मानव ट्राइजर को।

सभी उत्पादों में इसका उपयोग (इसकी शक्तियाँ)

एक अनिवार्य गुणवत्ता स्तंभ के रूप में (Constitution §11.4.169 में `helix_qa` सबमॉड्यूल को आवश्यक टेस्ट प्रकारों में से एक नामित किया गया है), HelixQA परिवार के हर उत्पाद को समान शक्तियाँ प्रदान करता है:

- **स्वायत्त QA सत्र**: एकल कमांड `helixqa autonomous --project ... --platforms android,desktop,web` LLM-प्लस-विज्ञान एजेंट को छोड़ देता है, जो बिना किसी मानवीय हस्तक्षेप के वास्तविक ऐप्स को कवरेज लक्ष्य की ओर संचालित करता है, रिपोर्ट, टिकट और वीडियो जनरेट करता है।
- **टेस्ट बैंक / सुइट्स**: YAML बैंक (राउंड-219 का न्यूनतम मानक ≥ 30), प्लेटफॉर्म-लक्षित, प्राथमिकता-आधारित, और लाइन-दर-लाइन उन दस्तावेज़ों से जुड़े हुए जिन्हें वे सत्यापित करते हैं।
- **कैप्चर किए गए सबूत**: स्क्रीनशॉट, लॉगकैट, वीडियो, स्टैक ट्रेस और पूरा टाइमलाइन —केंद्रीकृत और हर रिपोर्ट से जुड़े हुए, ताकि किसी भी निर्णय को बाद में दोबारा देखा और ऑडिट किया जा सके।
- **स्वतंत्र निर्णय (§11.4.141 स्वतंत्रता सिद्धांत)**: इसका LLM-संचालित `issuedetector` और विज्ञान ओरेकल चल रहे ऐप के व्यवहार का मूल्यांकन उस एजेंट से स्वतंत्र रूप से करते हैं जिसने इसे नेविगेट किया, जिससे उस क्लासिक गड़बड़ी से बचा जाता है जहाँ कोई सिस्टम अपने ही काम को सही ठहरा देता है।
- **गेट + म्यूटेशन रैचेट**: `make qa-all / make anti-bluff` और `challenges/scripts/helixqa_orchestrator_challenge.sh` (8 चरण, अंतर्निहित §1.1 म्यूटेशन) HelixQA की ईमानदारी को लगातार साबित करते रहते हैं—और जानबूझकर कोई `--skip-helixqa` शॉर्टकट नहीं रखा गया है ताकि समय सीमा के दबाव में अनुशासन को बंद न किया जा सके।

सामग्री

सबसे बड़ी तकनीकी चुनौतियाँ और उनके समाधान

- **गुणवत्ता जाँच में ही झूठी सकारात्मकता को रोकना** — वह उपकरण जो धोखे पकड़ता है, खुद धोखा न बन जाए → हर कदम पकड़े गए साक्ष्य के आधार पर सत्यापित किया जाता है, साक्ष्य के बिना पास को पास के बजाय दोष के रूप में दर्ज किया जाता है, और एक व्यवहार-एंकर मेनिफेस्ट हर घोषित क्षमता को एक निष्पादन योग्य परीक्षण (CONST-035) से जोड़ता है ताकि बिना परीक्षण के कोई क्षमता दावा न की जा सके।
- **एक ही मस्तिष्क से विविध प्लेटफॉर्म चलाना** — एंड्रॉयड, एंड्रॉयड टीवी, वेब और डेस्कटॉप का इनपुट मॉडल एक जैसा नहीं होता → एकल navigator पैकेज प्लेटफॉर्म-विशिष्ट ActionExecutors (ADB, Playwright, X11) और प्रति-प्लेटफॉर्म क्रैश डिटेक्टरों (एंड्रॉयड/वेब/डेस्कटॉप) को अमूर्त करता है, ताकि ऑर्केस्ट्रेशन लॉजिक एक बार लिखा जाए और प्लेटफॉर्म के अंतर किनारों पर ही रहें।
- **स्वायत्त एजेंटों को उपयोगी बनाना, अराजक नहीं** — एक अनियंत्रित LLM ऐप में हमेशा भटक सकता है → LLMsVerifier मॉडलों का मूल्यांकन और चयन करता है, LLMOrchestrator हेडलेस CLI एजेंटों (opencode, claude-code, gemini, junie, qwen-code) का प्रबंधन करता है, DocProcessor फीचर मैप बनाता है जो अन्वेषण को एक लक्ष्य देता है, और VisionEngine हर निर्णय को मॉडल की कल्पना के बजाय स्क्रीन पर वास्तविक पिक्सल के आधार पर रखता है।
- **स्थानीयकरण-सुरक्षित बैंक** — एक ऐसा सूट जो अंग्रेजी UI टेक्स्ट को हार्डकोड करता है, पंद्रह भाषाओं में टूट जाता है → बैंक केवल संरचना का वर्णन करते हैं, और उपयोगकर्ता-सामने आने वाला प्रॉम्प्ट टेक्स्ट LLM/रिसोर्स-लोडेड रनटाइम पर होता है (CONST-046), ताकि एक ही बैंक किसी भी स्थान पर समान व्यवहार की जाँच करे।
- **साबित करना कि गेट धोखा नहीं हैं** — एक ऐसा एंटी-ब्लफ़ गेट जो खुद विफल न हो सके वह अंतिम धोखा है → युग्मित §1.1 म्यूटेशन एक प्रकार के साक्ष्य-संग्रहण या एंटी-ब्लफ़ दावे को हटाते हैं और गेट को विफल होने के लिए बाध्य करते हैं, और एक म्यूटेशन रैचेट इस गारंटी को समय के साथ चुपचाप कमजोर होने से रोकता है।

तकनीकी स्टैक

- **Go 1.24+ ऑर्केस्ट्रेटर** — क्यों: गुणवत्ता जाँच को उन सभी जगहों पर चलना चाहिए जहाँ उत्पाद चलते हैं, इसलिए एकल स्टैटिकली-लिंकड, तेज़ और पोर्टेबल बाइनरी रनटाइम-भारी विकल्प से बेहतर है; कैसे: एक cmd/helixqa CLI जिसमें संयोज्य उप-कमांड run / list / report / autonomous / version शामिल हैं।
- **YAML टेस्ट बैंक (pkg/testbank)** — क्यों: सूट घोषणात्मक और पठनीय होने चाहिए, जिन्हें Go को छुए बिना इंसान संपादित कर सकें; कैसे: version/name/test_cases[] जिसमें id, category, priority, platforms, क्रमबद्ध steps[], और documentation_refs[] शामिल हैं ताकि फीचर दस्तावेजों से पता लगाया जा सके।
- **क्रैश/ANR डिटेक्टर (pkg/detector)** — क्यों: सबसे महत्वपूर्ण विफलताएँ वे होती हैं जो लाइव, इंटरैक्शन के दौरान होती हैं, न कि बाद में दावे के रूप में; कैसे: एंड्रॉयड के लिए ADB (pidof/logcat/screenshot) और वेब/डेस्कटॉप के लिए pgrep, जो प्रक्रिया को टेस्ट चलाते समय निगरानी करते हैं।

- **साक्ष्य संग्रह (pkg/evidence, pkg/session)** — क्यों: एंटी-ब्लफ़ अनुबंध तभी वास्तविक होता है जब हर पास के पीछे भौतिक प्रमाण हो; कैसे: स्क्रीनशॉट, logcat, वीडियो और स्टैक ट्रेस एक SessionRecorder टाइमलाइन में कैप्चर किए जाते हैं, जिससे हर रिपोर्ट जुड़ी होती है।
- **स्वायत्त सत्र (pkg/autonomous, pkg/navigator, pkg/issuedetector)** — क्यों: चार प्लेटफ़ॉर्म पर मैनुअल गुणवत्ता जाँच का विस्तार नहीं किया जा सकता, इसलिए अन्वेषण को खुद संचालित होना चाहिए; कैसे: एक 4-चरणीय SessionCoordinator प्लस ActionExecutors (ADB/Playwright/X11) और LLM बग डिटेक्शन जो विज़ुअल, UX, एक्सेसिबिलिटी और कार्यात्मक दोषों को कवर करता है।
- **बाहरी सबमॉड्यूल** — क्यों: पुनः उपयोग और डिकपलिंग (CONST-051), और — महत्वपूर्ण रूप से — नेविगेटर को जज से अलग करना; कैसे: LLMsVerifier (मॉडल स्कोरिंग), LLMOrchestrator (हेडलेस CLI एजेंट), VisionEngine (GoCV + LLM विज़न), DocProcessor (फीचर-मैप/कवरेज), प्रत्येक स्वतंत्र रूप से स्वामित्व वाला घटक।
- **एंटी-ब्लफ़ गेट + म्यूटेशन रैचेट** — क्यों: HelixQA को उसी §1.1 अनुबंध का पालन करना चाहिए जो वह दूसरों पर लागू करता है; कैसे: एक make anti-bluff स्कैन प्लस व्यवहार-एंकर मेनिफ़ेस्ट और म्यूटेशन रैचेट, जिसमें helixqa_orchestrator_challenge.sh एक 8-चरणीय एंड-टू-एंड वैलिडेटर के रूप में कार्य करता है।
- **15-पंक्ति कवरेज मैट्रिक्स (docs/test-coverage.md)** — क्यों: CONST-050(B) एक बंद, पूरी तरह से लेखांकित टेस्ट-टाइप सेट की मांग करता है जिसमें कोई अंतराल न हो; कैसे: हर पंक्ति एक ठोस निष्पादन योग्य संपत्ति और एक विशिष्ट साक्ष्य-स्वरूप से जुड़ी होती है, ताकि कवरेज एक जाँच योग्य तथ्य हो, न कि केवल दावा।

सामग्री

स्थिति एवं ईमानदारी संबंधी टिप्पणियाँ

- **स्थिति:** बीटा। सक्रिय रूप से विकसित (README स्थिति बैनर दौर 219)। अपने स्वयं के 'ब्लफ़-विरोधी' मानक पर खरा उतरता है।
- **लाइसेंस:** अपाचे-2.0। इंस्टॉल करें: `go install digital.vasic.helixqa/cmd/helixqa@latest`।

प्राथमिकता स्तर: Helix-प्राथमिक — Helix परिवार के लिए एक अनिवार्य गुणवत्ता/ब्लफ़-विरोधी स्तंभ, जो यह सत्यापित करता है कि विशेषताएँ वास्तव में कार्य करती हैं।